

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO FINAL - ESTACIONAMENTO CENTRAL

Sistema Web Orientado a Objetos em PHP

Discente: Gabriel Uaren Melo Sousa
Disciplina: Programação Web I
Professor: Dr. Renato William Rodrigues de Souza
Tema: 08 - Estacionamento
Repositório: github.com/GabrielUaren18/pweb1-trabalho-oo

Boa Viagem – CE

2026

1. Introdução e objetivos

O Estacionamento Central é um sistema web simples que registra estadias de veículos, calcula o valor a pagar e apresenta um histórico dos registros. O projeto foi desenvolvido individualmente para aplicar os conteúdos de HTTP, PHP e orientação a objetos estudados na disciplina Programação Web I.

O problema escolhido é comum: veículos de tipos diferentes podem ter preços diferentes por hora, e o responsável pelo estacionamento precisa calcular os valores sem fazer contas manualmente. O sistema atende a essa situação com uma interface de cadastro e cálculo.

Objetivos

Cadastrar uma estadia com tipo de veículo, placa, modelo e quantidade de horas; aplicar a tarifa correspondente; recusar informações inválidas; listar as estadias e somar o valor arrecadado. O sistema também oferece uma resposta JSON e grava os registros em arquivo para mantê-los entre requisições.

Escopo

O projeto é um protótipo acadêmico. Não inclui autenticação, controle de vagas, pagamentos reais ou integração com banco de dados. As tarifas são valores de exemplo definidos nas classes.

Organização do projeto

Local	Responsabilidade
src/	Classes de veículos, estadias e estacionamento
public/index.php	Rotas, processamento do formulário e resposta JSON
public/estilo.css	Apresentação visual das páginas
teste.php	Testes executados no terminal
data/estadias.json	Persistência local (arquivo gerado na execução)
docs/	Modelagem, testes, diário de IA e relatório

2. Requisitos e regras de negócio

As funcionalidades principais são o cadastro de estadias pelo formulário POST, o cálculo de tarifas, a visualização do histórico, a consulta em JSON e a manutenção dos registros em arquivo.

Tarifas usadas no projeto

Tipo	Tarifa por hora	Exemplo
Carro	R$ 6,00	3 horas = R$ 18,00
Moto	R$ 3,00	2 horas = R$ 6,00
Caminhonete	R$ 8,00	2 horas = R$ 16,00

Regras de negócio

Regra 1: placa e modelo não podem ser vazios. O construtor de Veiculo recusa os dados com uma exceção.

Regra 2: a duração da estadia deve ser maior que zero. Valores nulos ou negativos são recusados pelo método encerrar().

Regra 3: uma estadia já encerrada não pode ser encerrada outra vez. Isso evita que o mesmo objeto tenha seu tempo alterado depois do encerramento.

Regra 4: o nome do estacionamento não pode ser vazio.

Regra 5: o método setModelo() não permite substituir o modelo por texto vazio.

Tratamento das recusas

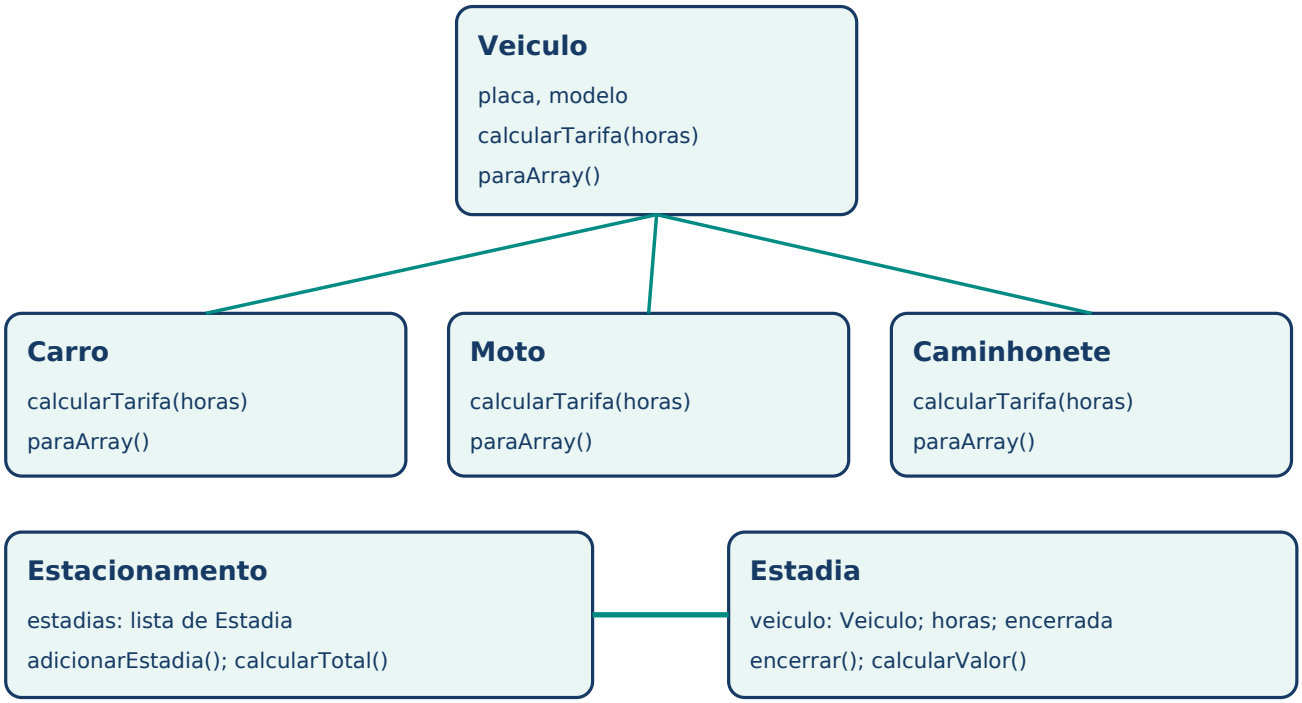
```
if ($horas <= 0) {
    throw new Exception("O tempo deve ser maior que zero.");
}
```

As exceções são capturadas por blocos try/catch, tanto nos testes de terminal quanto no processamento do formulário, para exibir uma mensagem compreensível em vez de interromper a página.

3. Modelagem de classes

O sistema possui seis classes: Veiculo, Carro, Moto, Caminhonete, Estadia e Estacionamento. Veiculo concentra as informações comuns; as três classes filhas diferenciam o cálculo da tarifa. Uma Estadia associa um veículo a um tempo e a um valor. Estacionamento mantém uma coleção de estadias e calcula o total.

Diagrama simplificado



As linhas superiores representam herança; a ligação inferior representa o estacionamento reunindo estadias. Cada estadia guarda uma referência a um veículo.

Por que usar herança?

Carro, Moto e Caminhonete são tipos de Veiculo. Todos têm placa e modelo, mas cada um aplica sua própria tarifa. A classe mãe evita repetir o cadastro e as validações.

Por que usar composição?

O Estacionamento precisa guardar várias estadias e calcular a soma delas. Por isso, ele possui um array de objetos Estadia e um método para adicionar cada novo registro.

4. Implementação: conceitos de orientação a objetos

OO1 - Classes e tipagem: as seis classes ficam em arquivos separados, com construtores, propriedades tipadas e declaração strict_types=1.

OO2 - Herança: Carro, Moto e Caminhonete usam extends Veiculo e chamam parent::__construct() para reaproveitar a inicialização de placa e modelo.

```
class Carro extends Veiculo {
    public function __construct(string $placa, string $modelo) {
        parent::__construct($placa, $modelo);
    }
    public function calcularTarifa(float $horas): float {
        return $horas * 6.00;
    }
}
```

OO3 - Polimorfismo: no teste.php, um foreach percorre veículos de tipos diferentes e chama calcularTarifa() em cada objeto. O método é o mesmo, mas o valor depende da classe.

```
foreach ($veiculos as $veiculo) {
    echo $veiculo->calcularTarifa(2);
}
```

OO4 - Encapsulamento: as propriedades são private ou protected, sem atributos public. Os getters permitem consulta; setModelo() valida antes de alterar o modelo.

OO5 - Exceções: condições inválidas disparam throw new Exception e são tratadas por try/catch.

OO6 - Composição: Estacionamento guarda objetos Estadia em um array; adicionarEstadia() inclui um objeto e calcularTotal() soma os valores.

OO7 - Conversão: paraArray() prepara os dados dos objetos para a resposta JSON.

OO8 - Arquivos: cada classe está em src/ e é carregada com require_once.

5. Camada web e comunicação HTTP

O arquivo public/index.php recebe as requisições, prepara os objetos e depois exibe o HTML. A navegação é feita por parâmetros na URL, mantendo cabeçalho e menu comuns.

Rota ou ação	Comportamento
/?pagina=inicio	Página inicial do sistema
/?pagina=veiculos	Formulário, resultado e histórico
/?pagina=sobre	Explicação resumida do projeto
/?pagina=inexistente	Resposta HTTP 404
/?formato=json	Resposta JSON com estadias e total
POST /?pagina=veiculos	Cria e registra uma estadia

Verificações com curl.exe

```
curl.exe -i "http://localhost:8000/?pagina=veiculos"
HTTP/1.1 200 OK
Content-type: text/html; charset=UTF-8

curl.exe -i "http://localhost:8000/?pagina=inexistente"
HTTP/1.1 404 Not Found

curl.exe -i "http://localhost:8000/?formato=json"
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

O formulário envia os dados por POST. Se houver uma regra recusada, a mensagem da exceção é apresentada na página. Os dados exibidos no HTML passam por htmlspecialchars(), reduzindo o risco de interpretar conteúdo digitado como marcação HTML.

Persistência opcional implementada

O sistema usa json_decode() para ler as estadias gravadas e file_put_contents() para atualizar o arquivo data/estadias.json. O histórico permanece disponível após atualizar a página.

6. Interface: cadastro e histórico

O formulário permite escolher o tipo do veículo, informar placa, modelo e tempo estacionado. Após o envio, o sistema calcula o valor e registra a estadia.

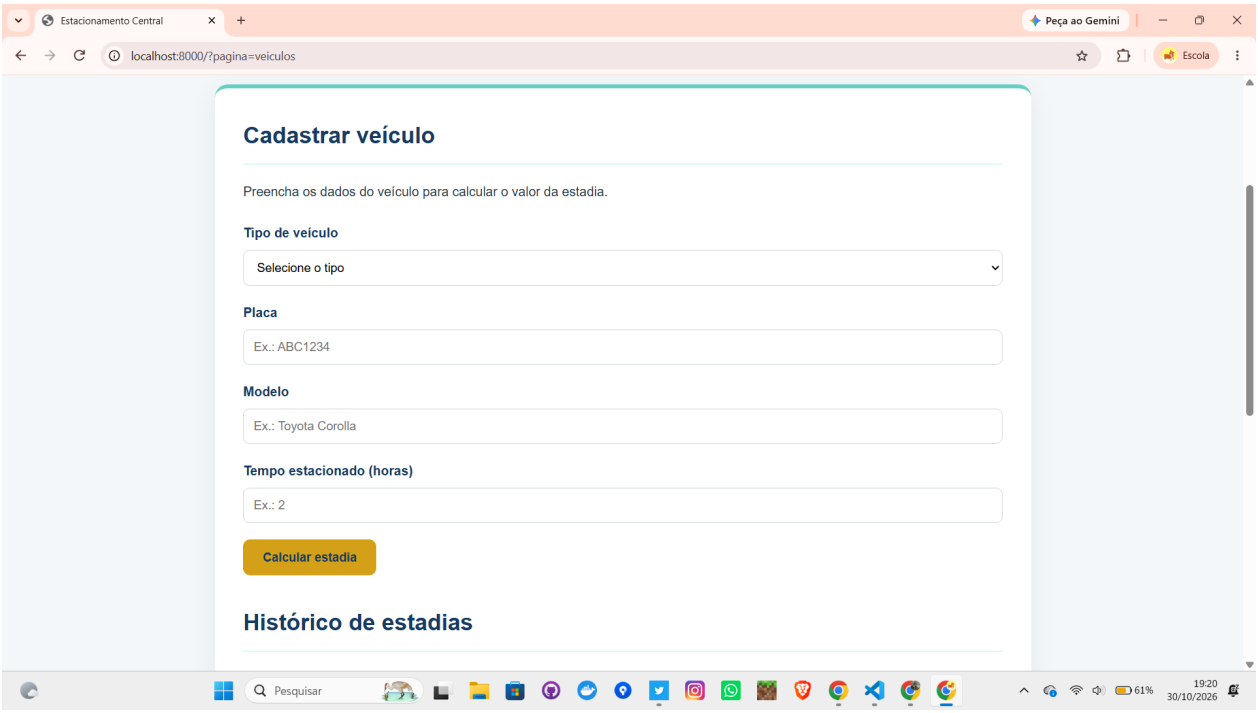


Figura 1. Formulário para cadastrar o veículo e informar o tempo estacionado.

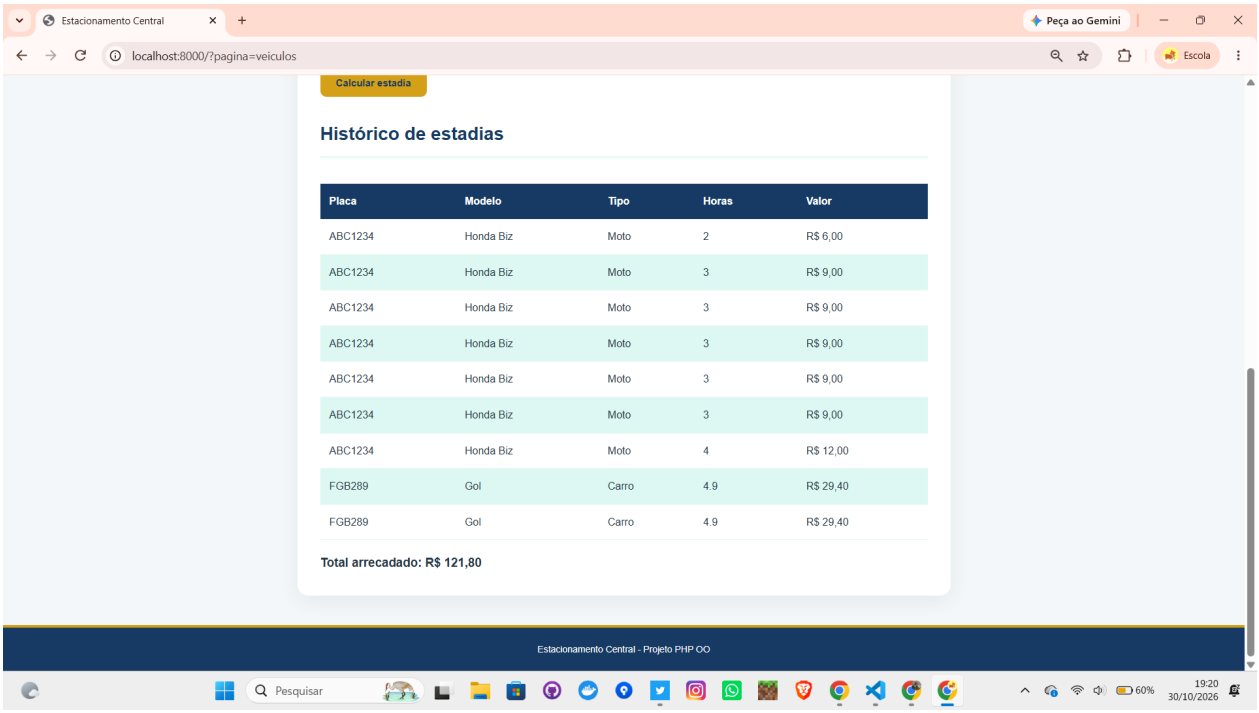


Figura 2. Histórico exibindo placa, modelo, tipo, horas e valor, com total arrecadado de R$ 121,80 no momento do registro.

7. Cadastro concluído e retorno JSON

Foi cadastrada uma caminhonete Toyota Hilux, placa XYZ9876, com duas horas de estadia. A aplicação confirmou o registro e calculou R$ 16,00, conforme a tarifa da classe Caminhonete.

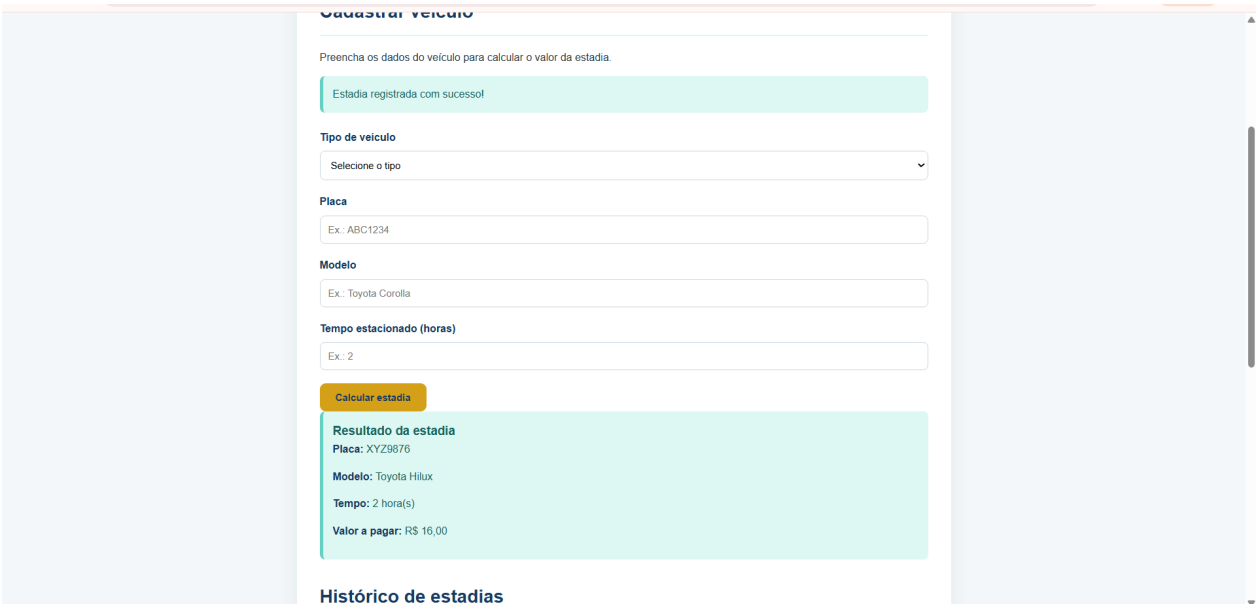


Figura 3. Mensagem de sucesso e valor calculado após o envio do formulário.

Resposta em JSON

```
{
  "nome": "Estacionamento Central",
  "estadias": [ ... ],
  "total": 121.8
}
```

Esse trecho representa a consulta realizada antes do cadastro da caminhonete. Naquele momento, havia nove estadias e total de R$ 121,80. Depois de novos cadastros, a lista e o total são atualizados.

O cálculo foi arredondado para duas casas decimais nas classes Estadia e Estacionamento, evitando valores como 121.80000000000001 na resposta.

8. Testes executados

O arquivo teste.php foi executado com php .\teste.php. Os nove testes passaram, cobrindo herança, polimorfismo, validações, composição, cálculo e conversão de objetos para array.

Teste	Resultado observado
1. Herança e polimorfismo	Carro R$ 12,00; Moto R$ 6,00; Caminhonete R$ 16,00
2. Placa vazia	OK, recusado: Placa e modelo são obrigatórios.
3. Modelo vazio	OK, recusado: Placa e modelo são obrigatórios.
4. Zero horas	OK, recusado: O tempo deve ser maior que zero.
5. Horas negativas	OK, recusado: O tempo deve ser maior que zero.
6. Encerramento repetido	OK, recusado: Esta estadia já foi encerrada.
7. Valor da estadia	R$ 18,00; OK, cálculo correto
8. Composição e total	R$ 34,00; OK, total correto
9. Conversão para array	OK, dados convertidos corretamente

Teste adicional do setter

```
Onix
OK, recusado: O modelo é obrigatório.
```

O teste adicional verificou que setModelo() altera o modelo quando recebe um texto válido e recusa uma string contendo apenas espaço.

Teste de formulário com dado inválido

Uma requisição POST com horas=0 retornou a mensagem "O tempo deve ser maior que zero.". O histórico permaneceu com a mesma quantidade de registros, confirmando que a estadia inválida não foi gravada.

9. Ferramentas, IA, dificuldades e conclusão

Ferramentas utilizadas

PHP 8.5: linguagem do servidor e das classes. VS Code: edição dos arquivos. PowerShell e curl.exe: execução e teste das requisições HTTP. Git e GitHub: versionamento, commits e publicação do repositório. Navegador: teste visual do formulário e do histórico. ChatGPT: apoio para entender conceitos, revisar mensagens de erro e organizar os testes e a documentação.

Uso de IA e avaliação crítica

A IA foi utilizada como apoio ao aprendizado, não como substituta da conferência do código. As interações relevantes estão descritas em docs/llm.md. Um exemplo de problema percebido durante o desenvolvimento foi a orientação de colocar a tabela de histórico em uma posição incorreta do fluxo condicional da página; a estrutura foi revisada para que o histórico aparecesse na rota de veículos. Outro problema envolveu o posicionamento de declare(strict_types=1), corrigido após investigar a mensagem de erro.

Dificuldades e aprendizados

As principais dificuldades foram entender a relação entre objetos, interpretar erros de PHP no terminal, posicionar corretamente a lógica antes do HTML, manter os registros após recarregar a página e lidar com diferenças de precisão nos números decimais. Os testes por etapas ajudaram a identificar cada problema sem alterar várias partes do projeto ao mesmo tempo.

Conclusão

O Estacionamento Central cumpre o objetivo de demonstrar orientação a objetos em um sistema web funcional. A hierarquia de veículos evita repetição, a composição organiza as estadias, as exceções protegem as regras de negócio e a interface web permite testar o comportamento. A persistência em JSON amplia o projeto ao manter os dados entre requisições. Como evolução futura, seria possível acrescentar horários reais de entrada e saída, autenticação e banco de dados.

Referências do próprio projeto

Enunciado: Trabalho individual - Sistema Web Orientado a Objetos em PHP, Prof. Renato William Rodrigues de Souza (2026).

Código-fonte e documentação: github.com/GabrielUaren18/pweb1-trabalho-oo.